Lab Report: Lab 7 - Event Driven Architecture

Kyte

Contents

1. Cover Page
2. Abstract/Summary
3. Lab Specific Section: VII. Event Driven Architecture 3.1 Objectives 3.2 Technology and Tools 3.3 Event Catalog 3.4 Producers and Consumers 3.5 Real Time Flow
4. Conclusion & Reflection

1. Cover Page

Title: Event Driven Architecture for the Kyte Project Management Tool

Course Info: Software Architecture

Student Details:

Họ và Tên	MSSV
Trần Thành Long	23010070
Nguyễn Xuân Mạnh	23010045

Date: Dec 17, 2025

2. Abstract/Summary

This lab documents how Kyte applies event driven architecture to decouple side effects such as notifications and real time updates from core task operations.

3. Lab Specific Section: VII. Event Driven Architecture

Note: Kyte uses an in memory event bus inside the monolith. There is no external message broker in the current deployment.

3.1 Objectives

- Define domain events for key actions.
- Use pub sub to decouple producers from consumers.
- Keep API response times fast by handling side effects asynchronously.

3.2 Technology and Tools

Tool	Purpose
Event Bus	Internal pub sub dispatcher.
WebSocket	Real time broadcasts to connected clients.
Resend	Email notifications as a consumer.

3.3 Event Catalog

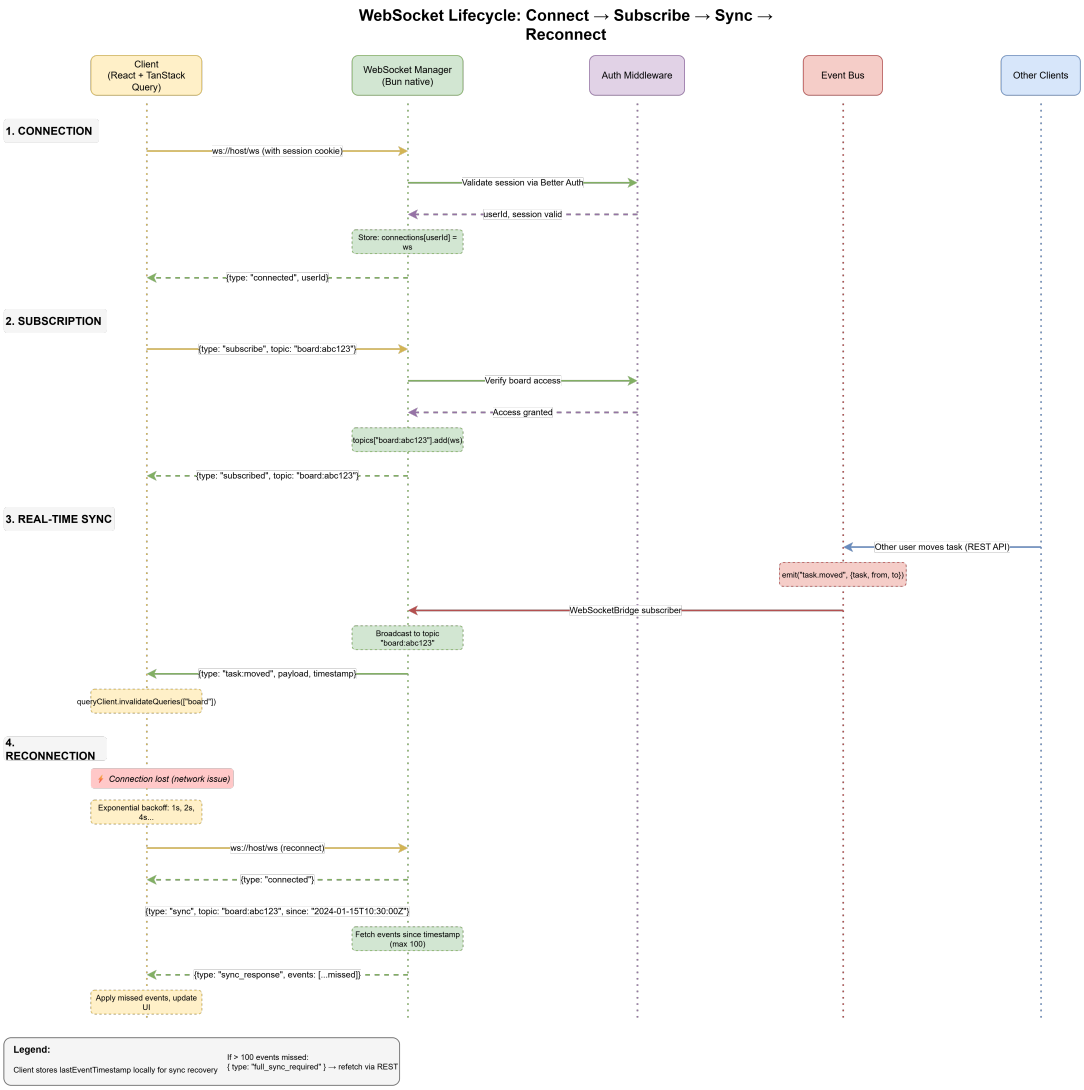
Event	Producer	Consumers
`task.created`	Task service	WebSocket, Activity log, Notifications
`task.moved`	Task service	WebSocket, Activity log
`comment.created`	Comment service	Notifications, Mentions

3.4 Producers and Consumers

- **Producers:** Services emit events after successful persistence.
- **Consumers:** WebSocket broadcaster, notification handler, and activity logger subscribe to events.

3.5 Real Time Flow

1. Task service persists a change and emits `task.moved`.
2. Event bus notifies WebSocket subscribers for the board.
3. Clients update UI in real time without polling.



4. Conclusion & Reflection

Event driven design keeps modules loosely coupled and improves responsiveness. It allows new features to be added as consumers without modifying existing service logic.